

Manual de usuario DREAM ML

Índice

Tabla de contenido

Índice	2
Introducción	1
1.1 Objetivo del manual	1
1.2 Descripción general del sistema.....	1
Requisitos previos.....	2
2.1 Requisitos de hardware y software	2
2.2 Herramientas y dependencias necesarias.....	2
2.3 Conocimientos previos.....	2
Instalación y configuración	3
3.1 Configuración inicial	3
3.2 Configuración de variables de entorno	3
3.1 Verificación de la instalación	4
Descripción general del sistema	6
4.1 Herramientas integradas	6
4.1.1 MLflow	6
4.1.2 DVC	11
4.2 Componentes del sistema.....	11
Guía de uso: Inicializar sistemas.....	12
5.1 Crear experimento	12
5.2 Inicializar DVC y Git	13
5.3 Configurar almacenamiento remoto de DVC	14
5.4 MLflow server.....	15
Guía de uso: Flujo manual	16
6.1 Ingesta y limpieza de datos.....	16
6.2 Generación de EDA	20
6.2.5 Paso opcional: EDA manual.....	22
6.3 Codificación de datos.....	23
6.4 Entrenamiento y validación	26

6.5 Generar reporte del experimento.	31
Guía de uso: Pipeline automatizado.....	33
7.1 Ejecución del pipeline	33
Glosario	36
Contacto	36

Introducción

1.1 Objetivo del manual

El objetivo de este manual es proporcionar una guía clara y detallada para la instalación, configuración y uso del sistema de gestión de experimentos **DREAM ML**. Este sistema está diseñado para mejorar la *reproducibilidad* y *documentación* de los experimentos de machine learning en entornos académicos, facilitando tanto el flujo manual de experimentos como la ejecución de un pipeline. Con este manual se busca que el usuario pueda:

- Instalar y configurar el entorno del sistema utilizando Docker.
- Comprender y ejecutar cada uno de los pasos clave del proceso: Ingesta y limpieza de datos, generación de EDA, codificación y entrenamiento y validación.

1.2 Descripción general del sistema

El sistema de gestión de experimentos **DREAM ML** se ha desarrollado para automatizar y sistematizar el ciclo de vida de los experimentos de machine learning desde la ingesta de datos hasta la validación del modelo en entornos académicos. Sus principales características son:

- **Flujo manual y Pipeline automatizado:** permite la ejecución manual de cada paso del proceso, junto con la posibilidad de ejecutar un pipeline que orquesta de forma secuencial las etapas del proceso.
- **Integración de herramientas especializadas:** se emplea *DVC* para el versionado de datos y *MLflow* para el registro y seguimiento de cada experimento, lo que garantiza la trazabilidad y reproducibilidad de los resultados.
- **Implementación en entorno Docker:** el sistema se distribuye en contenedores *Docker*, lo que simplifica su instalación, configuración y despliegue en diversos entornos.

Requisitos previos

2.1 Requisitos de hardware y software

- **Hardware:** se recomienda contar con recursos suficientes para ejecutar procesos de machine learning, pero se espera que el testing se realice en diversas plataformas para evaluar el rendimiento.
- **Software:** el sistema se ejecuta en contenedores *Docker*, lo que garantiza portabilidad y compatibilidad en diferentes sistemas operativos.

2.2 Herramientas y dependencias necesarias

- **Docker:** utilizado para la virtualización y despliegue del sistema.
- **Git:** para la gestión del repositorio de experimentos.
- **Python:** versión 3.8 o superior, aunque todas las dependencias necesarias se instalarán automáticamente a través del archivo `docker.tar` proporcionado.

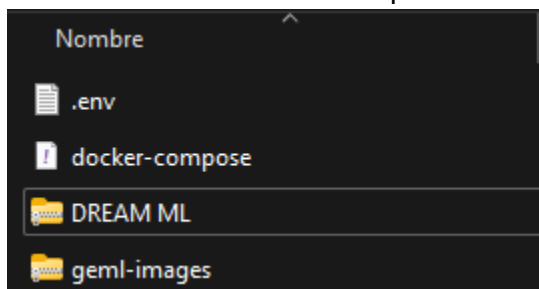
2.3 Conocimientos previos

- **Python:** para comprensión básica.
- **Uso de Docker:** para desplegar contenedores.
- **Machine Learning:** con énfasis en modelos de clasificación.

Instalación y configuración

3.1 Configuración inicial

1. Descarga el archivo ZIP **DREAM ML** desde el enlace de Google Drive proporcionado.
2. Extrae el contenido del ZIP **DREAM ML** en una carpeta local de tu elección.



3.2 Configuración de variables de entorno

1. Abre la carpeta extraída y localiza el archivo **.env** y el archivo **docker-compose.yml**.
2. Crea una carpeta en tu sistema que servirá como directorio donde se guardarán tus experimentos. Por ejemplo, crea una carpeta llamada **Experimentos** en tu directorio de usuario o en otra ubicación de preferencia.



3. Abre el archivo **.env** con un editor de texto y verás algo similar a lo siguiente:

```
# Ruta interna fija en el contenedor donde se almacenarán los experimentos
EXPERIMENTS_DIR=/app/experimentos

# Ruta en el host donde el usuario quiere guardar los experimentos.
# El usuario debe modificar este valor para apuntar a la carpeta deseada (por ejemplo, en Escritorio o Documentos).

HOST_EXPERIMENTS_DIR=C:/Users/espín/Desktop/Experimentos

#Recuerda invertir las parentésis tal como aparece en el ejemplo.
```

4. Edita la variable que define el directorio de experimentos **HOST_EXPERIMENTS_DIR** y reemplaza su valor con la ruta completa de la carpeta que acabas de crear.

5. Verifica que la ruta en ***HOST_EXPERIMENTS_DIR*** esté escrita con barras diagonales hacia adelante (/) y coincida exactamente con la carpeta que creaste en tu host.

Recuerda usar barras diagonales hacia adelante (/)

C:\Users\espin\Desktop\Experimentos



C:/Users/espin/Desktop/Experimentos

3.1 Verificación de la instalación

1. Abre una terminal en el directorio donde extrajiste el ZIP.
2. Ejecuta el siguiente comando para cargar la imagen Docker:

docker load -i geml-images.tar

```
C:\Users\espin\Desktop\App-test>docker load -i geml-images.tar|
```

3. Este comando importará las imágenes necesarias:

```
Loaded image: geml-frontend:latest
Loaded image: geml-backend:latest
```

```
C:\Users\espin\Desktop\App-test>|
```

4. Una vez cargada la imagen, en la misma terminal, inicia el sistema ejecutando:
docker-compose up

```
C:\Users\espin\Desktop\App-test>docker-compose up|
```

- Una vez los contenedores estén en ejecución:

```
[+] Running 3/3
✓Network app-test_default      Created
✓Container app-test-frontend-1 Created
✓Container app-test-backend-1  Created
Attaching to backend-1, frontend-1
frontend-1 |
frontend-1 | > frontend@0.0.0 dev
frontend-1 | > vite --host
frontend-1 |
frontend-1 | Re-optimizing dependencies because vite config has changed
frontend-1 |
frontend-1 | VITE v5.4.14 ready in 300 ms
frontend-1 |
frontend-1 | → Local:   http://localhost:5173/
frontend-1 | → Network: http://172.20.0.2:5173/
backend-1  | INFO:      Started server process [1]
backend-1  | INFO:      Waiting for application startup.
backend-1  | INFO:      ASGI 'lifespan' protocol appears unsupported.
backend-1  | INFO:      Application startup complete.
backend-1  | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

- Abre tu navegador y visita la URL configurada: <http://localhost:5173/>

[INICIO](#)
[INICIALIZAR SISTEMAS](#)
[FLUJO DEL EXPERIMENTO](#)
[EJECUCIÓN DEL PIPELINE](#)
[MANUAL DE USUARIO](#)

Bienvenido a DREAM ML

DREAM ML es un sistema enfocado en problemas de clasificación que centraliza todas las etapas clave desde la ingesta y limpieza de datos, hasta el entrenamiento y validación de tus modelos. Está diseñado para entornos académicos, facilitando la reproducibilidad y documentación mediante:

- **Versionado de datos y modelos con DVC:** garantiza la trazabilidad de cada conjunto de datos y modelo entrenado.
- **Registro de experimentos con MLflow:** almacena parámetros, métricas y artefactos de cada ejecución (*run*), facilitando la comparación de resultados.
- **Automatización o flujo manual:** elige entre ejecutar cada paso de forma independiente o reproducir todo el proceso a través de un pipeline secuencial.
- **EDA (Análisis Exploratorio) automático y manual:** genera reportes con herramientas como Ydata Profiling y Sweetviz, o profundiza con tu propio Jupyter Notebook.
- **Codificación de datos para clasificación:** soporta LabelEncoder y OneHotEncoder, adaptándose a modelos como Regresión Logística, MLP o XGBoost.
- **Reportes de validación y resultados:** obtén un PDF consolidado con métricas (Accuracy, F1, ROC-AUC, etc.) y detalles del proceso.

Con DREAM ML puedes organizar tus experimentos de clasificación de manera transparente. Explora las pestañas superiores para inicializar los sistemas (Git, DVC y MLflow), cargar tus datos, generar análisis exploratorios, codificar los atributos y entrenar tus modelos. Asegura la reproducibilidad y mantén un registro detallado de cada paso en tus experimentos de clasificación.

© 2025 DREAM ML

✉ Correo: leonardo.espinosa.o@usach.cl

☎ Celular: +56 9 7371 1546

🌐 [linkedin.com/in/leonardo-espinosa-ortiz-311229263/](https://www.linkedin.com/in/leonardo-espinosa-ortiz-311229263/)

📄 github.com/lespinozaortiz

- El sistema estará listo para usarse y los experimentos se guardarán en la carpeta que configuraste en tu host.

Descripción general del sistema

4.1 Herramientas integradas

El sistema incorpora dos herramientas fundamentales para la gestión de experimentos:

4.1.1 MLflow

Se utiliza principalmente para registrar cada **ejecución (run)** del experimento, en cada *run* se capturan:

- **Configuraciones:** parámetros de entrada y ajustes del experimento.
- **Métricas:** resultados obtenidos durante la ejecución de un *run*.
- **Artefactos:** modelos generados, gráficos, informes y otros archivos relevantes.

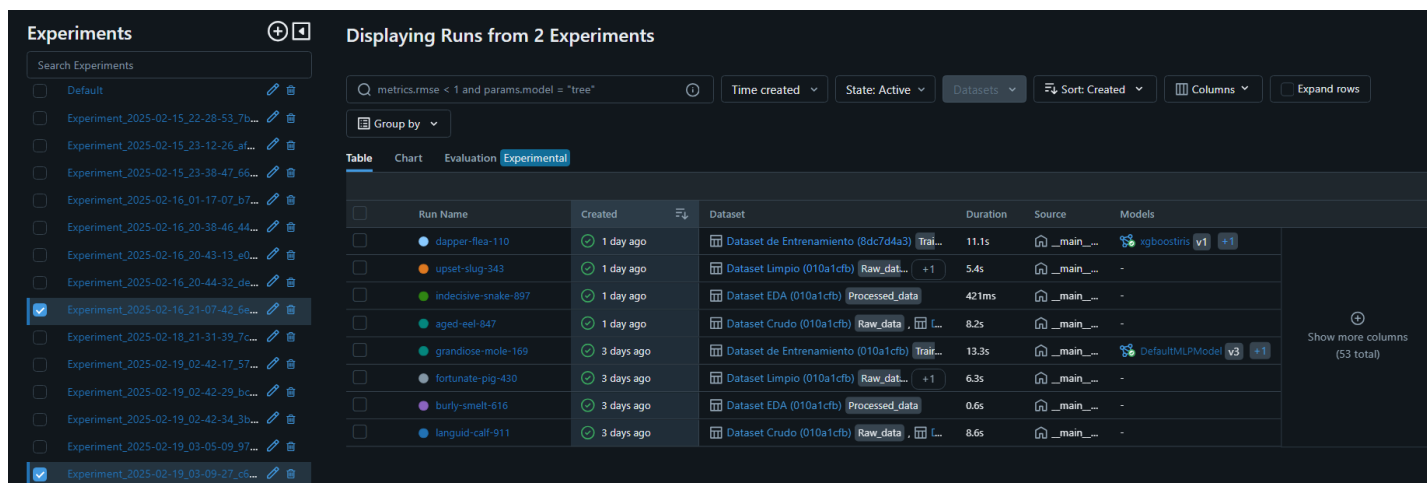
Estas capturas permiten:

- **Comparación entre diferentes runs.**
- **Trazabilidad de cambios y ajustes realizados en cada experimento.**
- **Versionado de modelos.**
- **Generación de reportes detallados basados en los resultados registrados.**

MLflow cuenta con una interfaz que permite visualizar las ejecuciones registradas de cada experimento:

The screenshot displays the MLflow web interface. On the left, under the 'Experiments' tab, there is a search bar and a list of experiments. A green arrow points from the 'Default' experiment in this list to a green box labeled 'Lista de experimentos'. The main area shows the 'Default' experiment details. At the top, there are filters for metrics (e.g., 'metrics.rmse < 1') and parameters (e.g., 'params.model = "tree"'). Below this is a table with columns: Run Name, Created, Dataset, Duration, Source, and Models. A green box labeled 'Información del run registrado del experimento.' points to the table header. At the bottom of the table area, a message states 'No runs logged' with a link to learn more.

- Puedes marcar la casilla de más de un experimento para ver sus runs registradas y así comparar métricas entre diferentes experimentos.



Experiments

Search Experiments

- ☐ Default
- ☐ Experiment_2025-02-15-22-28-53_7b...
- ☐ Experiment_2025-02-15-23-12-26_af...
- ☐ Experiment_2025-02-15-23-38-47_66...
- ☐ Experiment_2025-02-16-01-17-07_b7...
- ☐ Experiment_2025-02-16-20-38-46_44...
- ☐ Experiment_2025-02-16-20-43-13_e0...
- ☐ Experiment_2025-02-16-20-44-32_de...
- ☒ Experiment_2025-02-16-21-07-42_6e...
- ☐ Experiment_2025-02-18-21-31-39_7c...
- ☐ Experiment_2025-02-19-02-42-17-57...
- ☐ Experiment_2025-02-19-02-42-29_b...
- ☐ Experiment_2025-02-19-02-42-34_3b...
- ☐ Experiment_2025-02-19-03-05-09_97...
- ☒ Experiment_2025-02-19-03-09-27_c6...

Displaying Runs from 2 Experiments

Q metrics.rmse < 1 and params.model = "tree" Time created State: Active Datasets Sort: Created Columns Expand rows

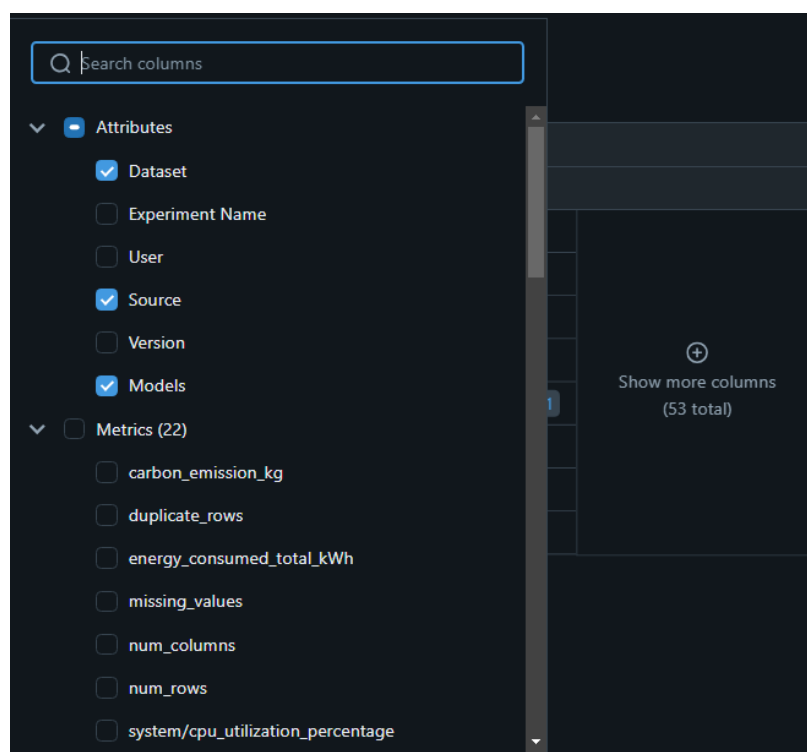
Group by

Table Chart Evaluation **Experimental**

Run Name	Created	Dataset	Duration	Source	Models
dapper-flea-110	1 day ago	Dataset de Entrenamiento (8dc7d4a3) Train...	11.1s	_main_...	xgboostiris v1 +1
upset-slug-343	1 day ago	Dataset Limpio (010a1cfb) Raw_data... +1	5.4s	_main_...	-
indecisive-snake-897	1 day ago	Dataset EDA (010a1cfb) Processed_data	421ms	_main_...	-
aged-eel-847	1 day ago	Dataset Crudo (010a1cfb) Raw_data, f...	8.2s	_main_...	-
grandiose-mole-169	3 days ago	Dataset de Entrenamiento (010a1cfb) Train...	13.3s	_main_...	DefaultMLPModel v3 +1
fortunate-pig-430	3 days ago	Dataset Limpio (010a1cfb) Raw_data... +1	6.3s	_main_...	-
burly-smelt-616	3 days ago	Dataset EDA (010a1cfb) Processed_data	0.6s	_main_...	-
languid-calf-911	3 days ago	Dataset Crudo (010a1cfb) Raw_data, f...	8.6s	_main_...	-

Show more columns (53 total)

- Si presionas **Show more columns**, puedes agregar más columnas visualizando atributos, métricas, parámetros, tags, etc.



Search columns

Attributes

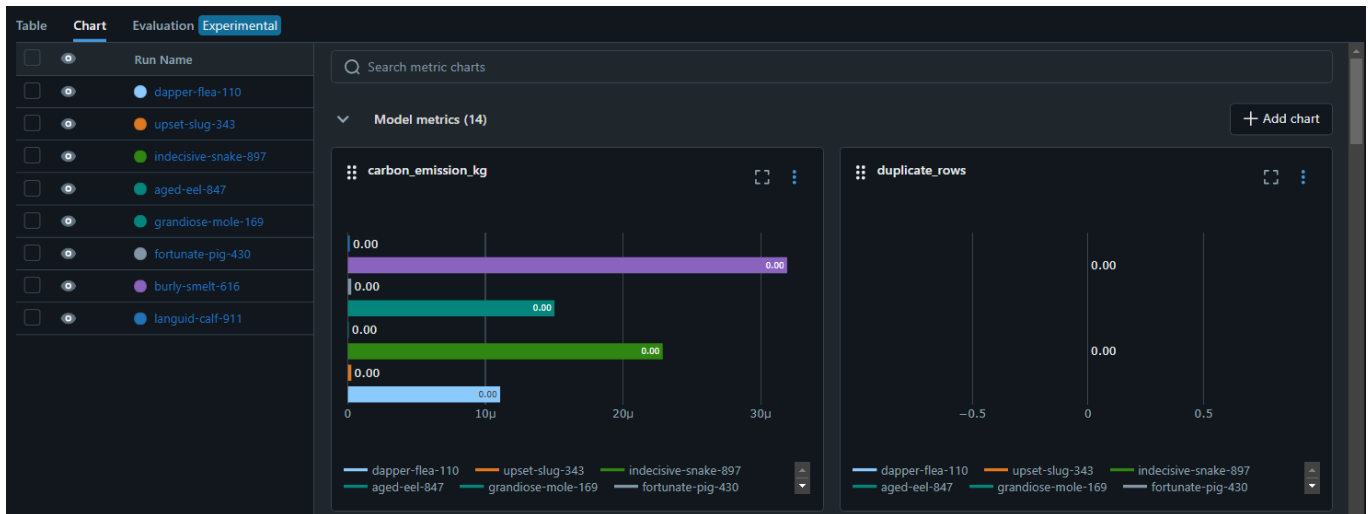
- ☒ Dataset
- ☐ Experiment Name
- ☐ User
- ☒ Source
- ☐ Version
- ☒ Models

Metrics (22)

- ☐ carbon_emission_kg
- ☐ duplicate_rows
- ☐ energy_consumed_total_kWh
- ☐ missing_values
- ☐ num_columns
- ☐ num_rows
- ☐ system/cpu_utilization_percentage

Show more columns (53 total)

- Si presionas **Chart**, puedes comparar diferentes métricas entre runs, por ejemplo, acá se puede visualizar que la ejecución “*buffy-smelt-616*” fue la que más generó emisiones de carbono.



- Si marcas la casilla de dos o más ejecuciones, puedes presionar el botón **compare** para comparar las métricas de las ejecuciones entre sí, esta función es bastante útil para comparar resultados de modelos entrenados, limpiezas, etc.

Rename

Delete

Compare

Table

Chart

Evaluation

Experimental

	Run Name	Created	Dataset	Duration	Source	Models	Parameters
☒	 dapper-flea-110	 1 day ago	 Dataset de Entrenamiento (8dc7d4a3) Trai...	11.1s	 _main_...	 xgboostiris v1 +1	xgboost_trai...
☐	 upset-slug-343	 1 day ago	 Dataset Limpio (010a1cfb) Raw_data... +1	5.4s	 _main_...	-	data_encodi...
☐	 indecisive-snake-897	 1 day ago	 Dataset EDA (010a1cfb) Processed_data	421ms	 _main_...	-	generate_eda
☐	 aged-eel-847	 1 day ago	 Dataset Crudo (010a1cfb) Raw_data , L...	8.2s	 _main_...	-	data_cleaning
☒	 grandiose-mole-169	 3 days ago	 Dataset de Entrenamiento (010a1cfb) Trai...	13.3s	 _main_...	 DefaultMLPModel v3 +1	mlp_training
☐	 fortunate-pig-430	 3 days ago	 Dataset Limpio (010a1cfb) Raw_data... +1	6.3s	 _main_...	-	data_encodi...
☐	 burly-smelt-616	 3 days ago	 Dataset EDA (010a1cfb) Processed_data	0.6s	 _main_...	-	generate_eda
☐	 languid-calf-911	 3 days ago	 Dataset Crudo (010a1cfb) Raw_data , L...	8.6s	 _main_...	-	data_cleaning

- Si presionas el nombre de un *run*, puedes visualizar la información más detallada de dicho *run*, incluyendo métricas, métricas del sistema y artefactos asociados.

Experiment_2025-02-19_03-09-27_c6da7ae7-d7ab-45de-bed7-90acc2e32207 >

dapper-flea-110

[Overview](#) [Model metrics](#) [System metrics](#) [Artifacts](#)

Description 

Entrenamiento xgboost

Details

Created at	2025-02-19 00:11:42
Created by	root
Experiment ID	14 
Status	 Finished
Run ID	fc677497df844c1783cd6ab01a83e96f 
Duration	11.1s
Datasets used	 Dataset de Entrenamiento (8dc7d4a3) Train_data
Tags	training_phase: completed 
Source	 __main__.py
Logged models	 xgboost
Registered models	 xgboostiris v1

- En el apartado de **Models**, puedes visualizar los modelos registrados por el sistema.

mlflow 2.13.2 [Experiments](#) [Models](#)

Registered Models

Filter registered models by name or tags  

Name 	Latest version	Aliased versions	Created by	Last modified	Tags
DefaultMLPModel	Version 3			2025-02-16 18:09:04	task: classification
mlp	Version 1			2025-02-18 23:47:07	task: classification
mlpins	Version 3			2025-02-19 00:09:05	task: classification
xgboostiris	Version 1			2025-02-19 00:11:50	—

- Si guardas más de un modelo bajo el mismo nombre, te permitirá crear diferentes versiones del modelo que hayas entrenado. Así puedes realizar comparaciones entre distintas versiones de este.

mlpiris

Created Time: 2025-02-15 20:50:20 Last Modified: 2025-02-19 00:09:05

▼ Description [Edit](#)

Modelo MLP para multiclass clasificación (mlpiris).

▼ Tags

Name	Value	Actions
task	classification	✎ 🗑

Name Value [Add](#)

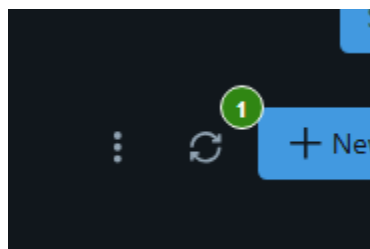
▼ Versions [Compare](#)

New model registry UI [🌙](#)

Version	Registered at	Created by	Tags	Aliases	Description
Version 3	2025-02-19 00:09:05		Add	Add	
Version 2	2025-02-19 00:07:55		Add	Add	
Version 1	2025-02-15 20:50:21		Add	Add	

1

- Puedes actualizar cada ejecución realizada presionando este botón en la interfaz de MLflow



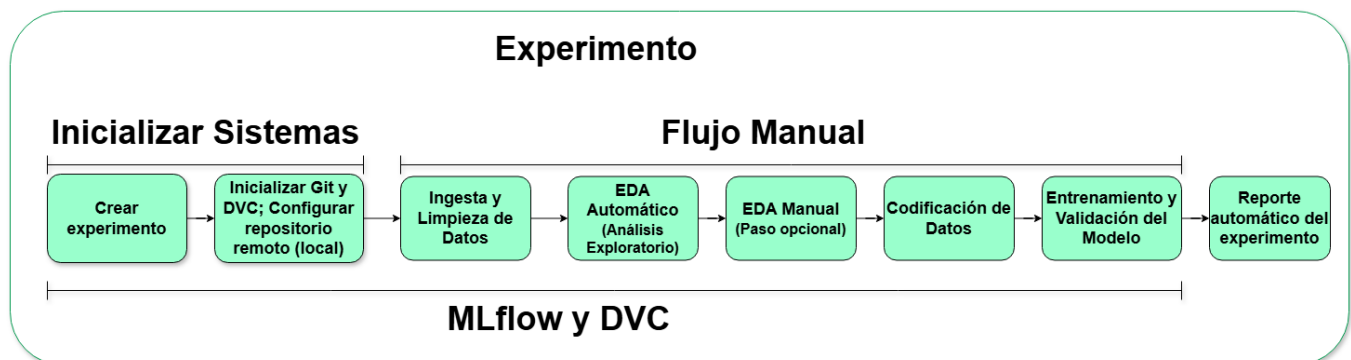
4.1.2 DVC

Se integra internamente en el flujo del experimento y en el pipeline. Su función es esencial para:

- **Ejecución del pipeline:** gestiona los archivos en la orquestación y secuenciación de las etapas del proceso.
- **Versiónado:** permite control de versiones de datasets y modelos, asegurando trazabilidad y reproducibilidad del experimento.

4.2 Componentes del sistema

Se presenta un diagrama general de los componentes del sistema junto con un breve glosario de cada uno:



- **Crear experimento:** punto de inicio donde se define un nuevo experimento, estableciendo su configuración base.
- **Inicializar Git y DVC:** configuración inicial del experimento para el versionado de datos, asegurando que se puedan rastrear los cambios a lo largo del tiempo.
- **Configurar DVC remoto:** definir y conectar el repositorio remoto DVC local, importante para compartir y mantener el historial de versiones.
- **Ingesta y limpieza de datos:** proceso de carga de datos y aplicación de técnicas de limpieza para eliminar inconsistencias, etc.
- **EDA automático:** generación de un análisis de datos utilizando herramientas como Ydata Profiling y Sweetviz, que ofrecen informes y visualizaciones automáticas.
- **EDA manual (opcional):** paso complementario que permite al usuario realizar un análisis más detallado y personalizado cuando sea necesario.
- **Codificación de datos:** paso que permite preparar los datos para el entrenamiento.
- **Entrenamiento y validación:** paso que permite entrenar modelos de machine learning y validar su desempeño mediante técnicas y métricas definidas.
- **Reporte automático del experimento:** generación de reporte que consolida los resultados, métricas y visualizaciones del experimento.

Guía de uso: Inicializar sistemas

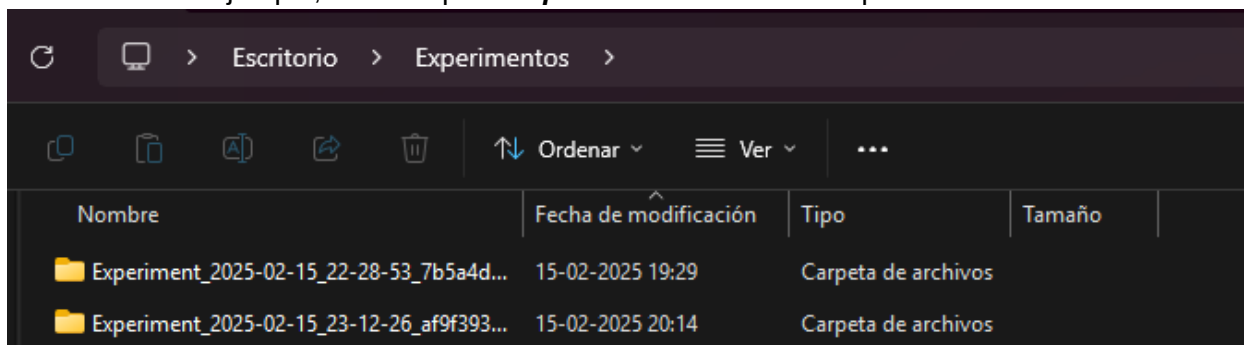
Esta sección describe los pasos iniciales para poner en marcha el sistema y preparar el entorno para la ejecución de experimentos.

5.1 Crear experimento

Al presionar **Crear Experimento**, el sistema valida la ruta definida, en caso de que no exista una base de datos compartida, crea una. Luego, genera una carpeta única en el directorio definido para el experimento, registra el experimento en *MLflow* y crea un archivo de configuración **pipeline_config.json** para ir registrando los pasos del pipeline.



- En este ejemplo, en la carpeta **Experimentos** se crean experimentos.



5.2 Inicializar DVC y Git

Al presionar **Inicializar DVC y Git**, el sistema verifica que exista la carpeta del experimento, inicializa un repositorio *Git* y ejecuta **dvc init** para configurar *DVC*. Además, crea la caché de *DVC* en el directorio *.dvc_cache* y gestiona el archivo *.gitignore* para excluir archivos innecesarios. Todo esto se realiza automáticamente para preparar el entorno de versionado de datos.



5.3 Configurar almacenamiento remoto de DVC

Al presionar **Configurar almacenamiento remoto de DVC**, el sistema crea un directorio compartido para *DVC* en la carpeta base del experimento, lo añade como remoto en *DVC* y lo establece como predeterminado. Esto permite respaldar y sincronizar las versiones de datos y modelos.



5.4 MLflow server

Al presionar **Iniciar MLflow**, el sistema arranca el servidor de *MLflow* en modo local usando la base de datos compartida y un directorio de artefactos definido en el directorio base del experimento. Una vez iniciado, el usuario puede abrir la interfaz de *MLflow* para visualizar y comparar los experimentos registrados.

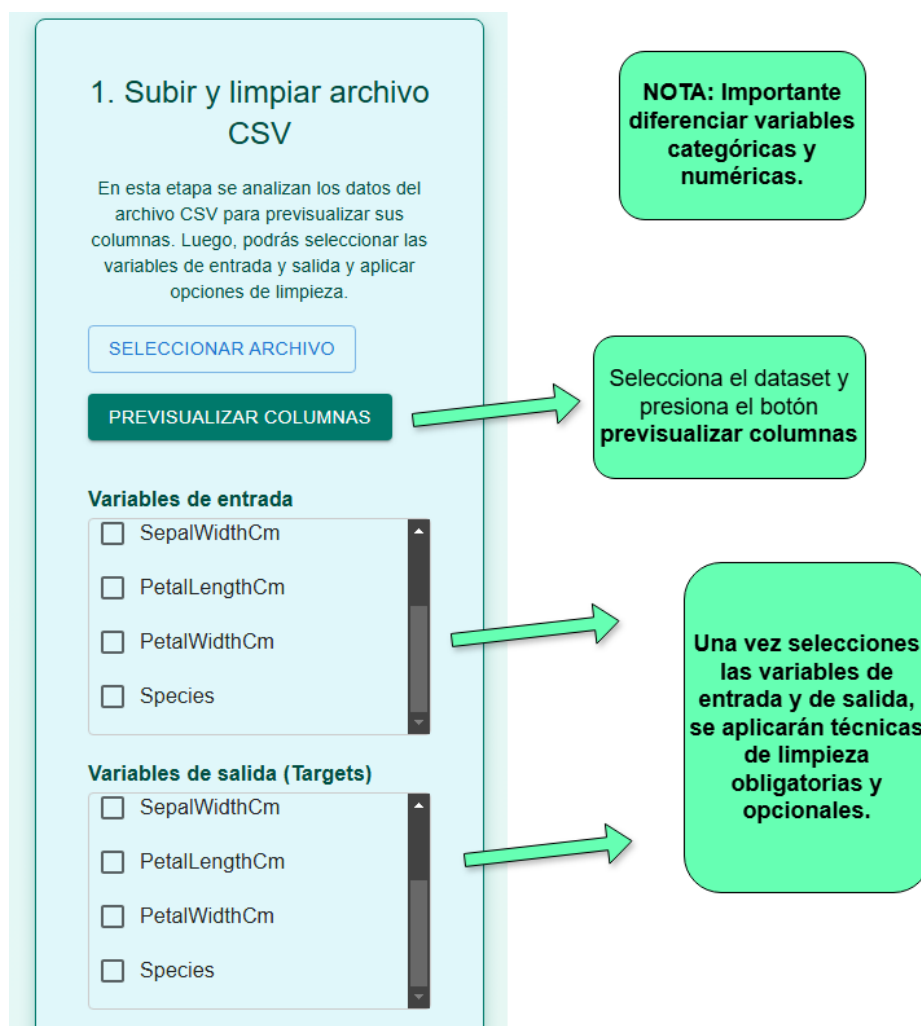


Guía de uso: Flujo manual

En el flujo manual, cabe mencionar que todas las ejecuciones registran la **energía consumida** y las **emisiones de carbono**.

6.1 Ingesta y limpieza de datos

Durante la ingesta y limpieza de datos, se aplican diversas técnicas para mejorar la calidad del CSV. Algunas de estas técnicas son obligatorias, mientras que otras se ejecutan opcionalmente según la configuración del usuario. A continuación, se detalla el proceso:



Técnicas obligatorias:

- **Limpieza de nombres de columnas:** se eliminan espacios adicionales en los nombres para evitar errores futuros.
- **Eliminación de espacios en celdas:** se remueven los espacios en blanco al inicio y al final de cada celda.
- **Reemplazo de valores vacíos por NaN:** se sustituyen celdas vacías o que contengan únicamente espacios por NaN, facilitando la detección de valores faltantes.
- **Conversión de columnas a tipo numérico:** se transforman columnas que pueden representar números (aunque estén almacenados como texto) para asegurar cálculos correctos.
- **Relleno de valores faltantes en columnas categóricas:** se reemplazan los valores faltantes en columnas de texto con palabra vacío
- **Eliminación de columnas completamente vacías:** se suprimen aquellas columnas que solo contienen valores nulos.

The screenshot shows a light blue interface with two checkboxes at the top: 'Eliminar duplicados' and 'Filtrar outliers'. Below them is a section titled 'Relleno de valores numéricos:' with a dropdown menu labeled 'Selecciona una opción'. The dropdown is open, showing four options: 'Dejar valores faltantes' (highlighted), 'Imputar con la media', 'Imputar con un valor', and 'Eliminar filas con valores nulos'. At the bottom, there is a text prompt: 'Selecciona las variables de entrada y salida.'

Marca la casilla para seleccionar si quieres aplicar la técnica opcional.

Para el relleno de valores numéricos, debes seleccionar una opción en el desplegable.

Técnicas opcionales:

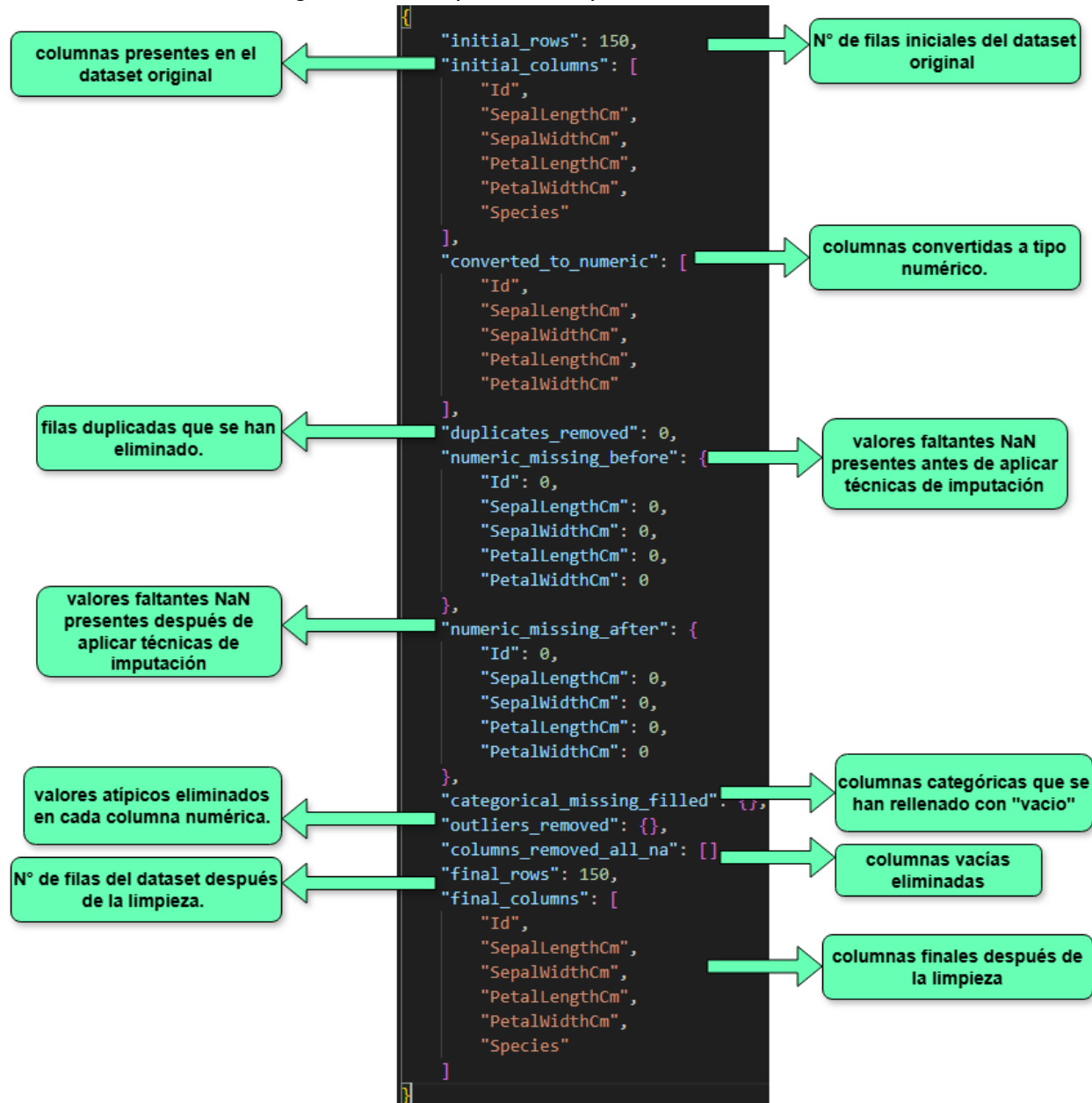
- **Eliminación de duplicados:** si se activa esta opción, se remueven filas duplicadas para evitar redundancias.
- **Filtrado de outliers:** si se activa esta opción, se eliminan los valores atípicos en las columnas numéricas basándose en el rango intercuartil (IQR).
- **Imputación de valores faltantes en columnas numéricas:**
 - **Imputar con la media:** reemplaza los valores nulos por el promedio de la columna.
 - **Imputar con un valor específico:** utiliza un valor definido por el usuario para rellenar las celdas vacías.
 - **Eliminar filas con valores nulos:** si selecciona la opción de eliminar, se suprimen las filas que contienen valores faltantes.
 - **Si se elije dejar, se mantienen los *NaN*, lo que puede causar problemas en algunos algoritmos.**

Finalmente, el dataset limpio se guarda en un nuevo archivo y se genera un reporte de limpieza que documenta cada transformación realizada. Este reporte se guarda en la carpeta **artifacts**, se registra en *MLflow* y se versiona mediante *DVC*, asegurando la trazabilidad del proceso.

Puedes revisar estos cambios en el directorio del experimento, en la carpeta **raw** se encuentra el archivo crudo y en la carpeta **processed** se encuentra el archivo limpio.

 .dvc	20-02-2025 21:51	Carpeta de archivos	
 .dvc_cache	20-02-2025 23:11	Carpeta de archivos	
 .git	20-02-2025 23:11	Carpeta de archivos	
 artifacts	20-02-2025 23:11	Carpeta de archivos	
 processed	20-02-2025 23:11	Carpeta de archivos	
 raw	20-02-2025 23:11	Carpeta de archivos	
 .dvcignore	20-02-2025 21:51	Archivo DVCIGNO...	1 KB
 .gitignore	20-02-2025 23:11	Archivo de origen ...	1 KB
 pipeline_config	20-02-2025 23:11	Archivo de origen ...	3 KB
 pipeline_config.json	20-02-2025 23:11	Archivo DVC	1 KB

A continuación, se ve un glosario del reporte de limpieza:



6.2 Generación de EDA

Al presionar **Generar Reportes**, el sistema ejecuta el proceso de *EDA* de forma automática sobre el archivo procesado. Este proceso genera dos reportes: uno usando *Ydata Profiling* y otro con *Sweetviz*.



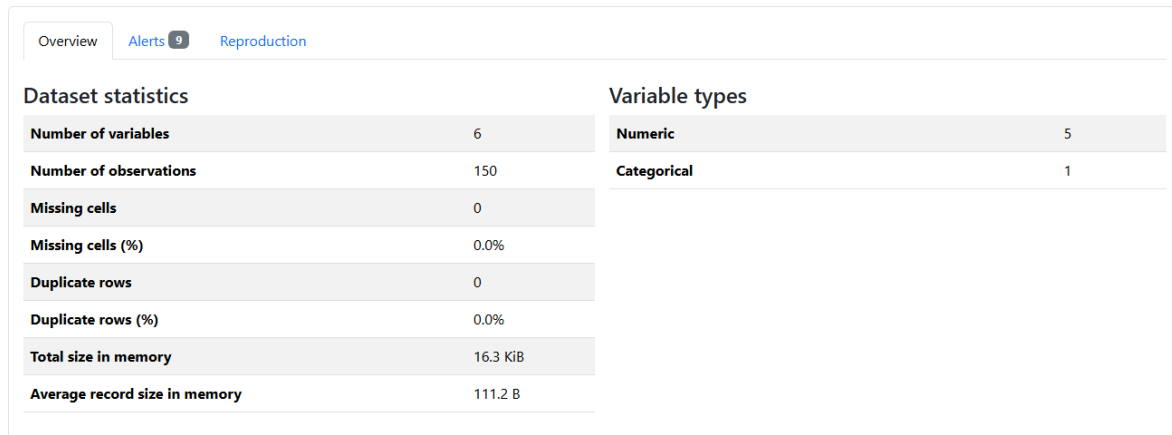
Puedes visualizar los reportes generados en la carpeta **eda_reports** del directorio de tu experimento y también en artifacts por MLflow:

Escritorio > Experimentos > Experiment_2025-02-21_03-36-43_bbad7618-fd08-4d38-acf9-eb6dfbd06649 >				
Ordenar Ver ...				
Nombre	Fecha de modificación	Tipo	Tamaño	
.dvc	21-02-2025 0:36	Carpeta de archivos		
.dvc_cache	21-02-2025 0:37	Carpeta de archivos		
.git	21-02-2025 0:38	Carpeta de archivos		
artifacts	21-02-2025 0:38	Carpeta de archivos		
eda_reports	21-02-2025 0:38	Carpeta de archivos		
processed	21-02-2025 0:37	Carpeta de archivos		
raw	21-02-2025 0:37	Carpeta de archivos		
.dvcignore	21-02-2025 0:36	Archivo DVCIGNO...	1 KB	
.gitignore	21-02-2025 0:37	Archivo de origen ...	1 KB	
pipeline_config	21-02-2025 0:38	Archivo de origen ...	3 KB	
pipeline_config.json	21-02-2025 0:38	Archivo DVC	1 KB	

Vista previa de los reportes:

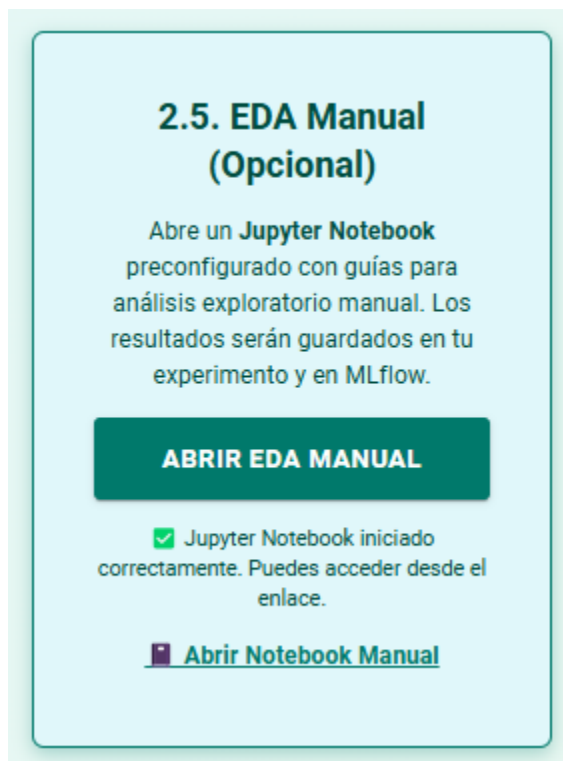
Overview

Brought to you by YData



6.2.5 Paso opcional: EDA manual

Al presionar **Abrir EDA Manual**, el sistema copia una plantilla de *Jupyter Notebook* (**EDA_manual.ipynb**) al directorio del experimento, inicia un servidor de Jupyter y registra este notebook en *MLflow*.



Este paso es opcional y está diseñado para complementar el EDA automático, ofreciendo un entorno donde el usuario pueda realizar análisis más avanzados, reducción de dimensionalidad (PCA, FAMD) y pruebas de hipótesis.

6.3 Codificación de datos

Al presionar **Codificar dataset** el sistema toma el archivo CSV seleccionado. Registra el dataset en *MLflow* y se aplica la codificación.

3. Codificación de Datos

Selecciona un CSV, define las variables y elige el método de codificación para targets.

SELECCIONAR ARCHIVO

PREVISUALIZAR COLUMNAS

📁 Archivo seleccionado.

Se recomienda que el usuario seleccione el archivo ***processed_eda*** ubicado en la carpeta ***processed***, ya que este archivo ha pasado por el proceso de limpieza y garantiza la consistencia del flujo de trabajo.

The image shows a web-based data preprocessing interface. It has two main sections: 'Variables de Entrada:' and 'Variables de Salida (Targets):'. The 'Variables de Entrada:' section has four checked checkboxes: SepalLengthCm, SepalWidthCm, PetalLengthCm, and PetalWidthCm. The 'Variables de Salida (Targets):' section has three unchecked checkboxes: SepalWidthCm, PetalLengthCm, and PetalWidthCm, and one checked checkbox: Species. Below these sections is a 'Opciones de Codificación para Targets (opcional):' section with two unchecked checkboxes: One-Hot Encoding and Label Encoding. At the bottom is a 'CODIFICAR DATASET' button and a checked checkbox labeled 'Columnas cargadas. Selecciona variables.'.

Annotations with arrows point to the interface:

- An arrow points from the 'Variables de Entrada:' section to a box: **Selecciona tus variables de entrada.**
- An arrow points from the 'Variables de Salida (Targets):' section to a box: **Selecciona tus variables de salida.**
- A box: **One-Hot Encoding: genera columnas binarias para cada categoría del target.**
- A box: **Label Encoding: transforma la variable categórica en etiquetas numéricas.**

Para variables de entrada:

- Se aplica ***get_dummies*** a columnas categóricas, lo que genera columnas binarias para cada categoría.
- Se utiliza el parámetro ***drop_first=True*** para descartar la primera columna, evitando la trampa de variables ficticias que se produce cuando las columnas generadas son linealmente dependientes y puede causar *problemas de multicolinealidad*.

Para variables de salida:

- Se ofrece la opción de codificarlas utilizando ***OneHotEncoder*** o ***LabelEncoder*** (estas opciones son mutuamente excluyentes).
- ***OneHotEncoder***: genera columnas binarias para cada categoría del *target*.
- ***LabelEncoder***: transforma la variable categórica en etiquetas numéricas.
- Esta elección es importante porque la elección depende del modelo a utilizar: por ejemplo, para ***XGBoost*** requiere utilizar ***LabelEncoder***, mientras que para un ***MLP*** puede ser preferible dejar la variable en su forma original.

Adicionalmente:

- Todas las columnas booleanas se convierten a enteros (0/1) para asegurar un correcto procesamiento en la etapa de entrenamiento.

En la siguiente tabla, se muestra el formato de *codificación* de los *datasets* para cada algoritmo en la etapa de entrenamiento.

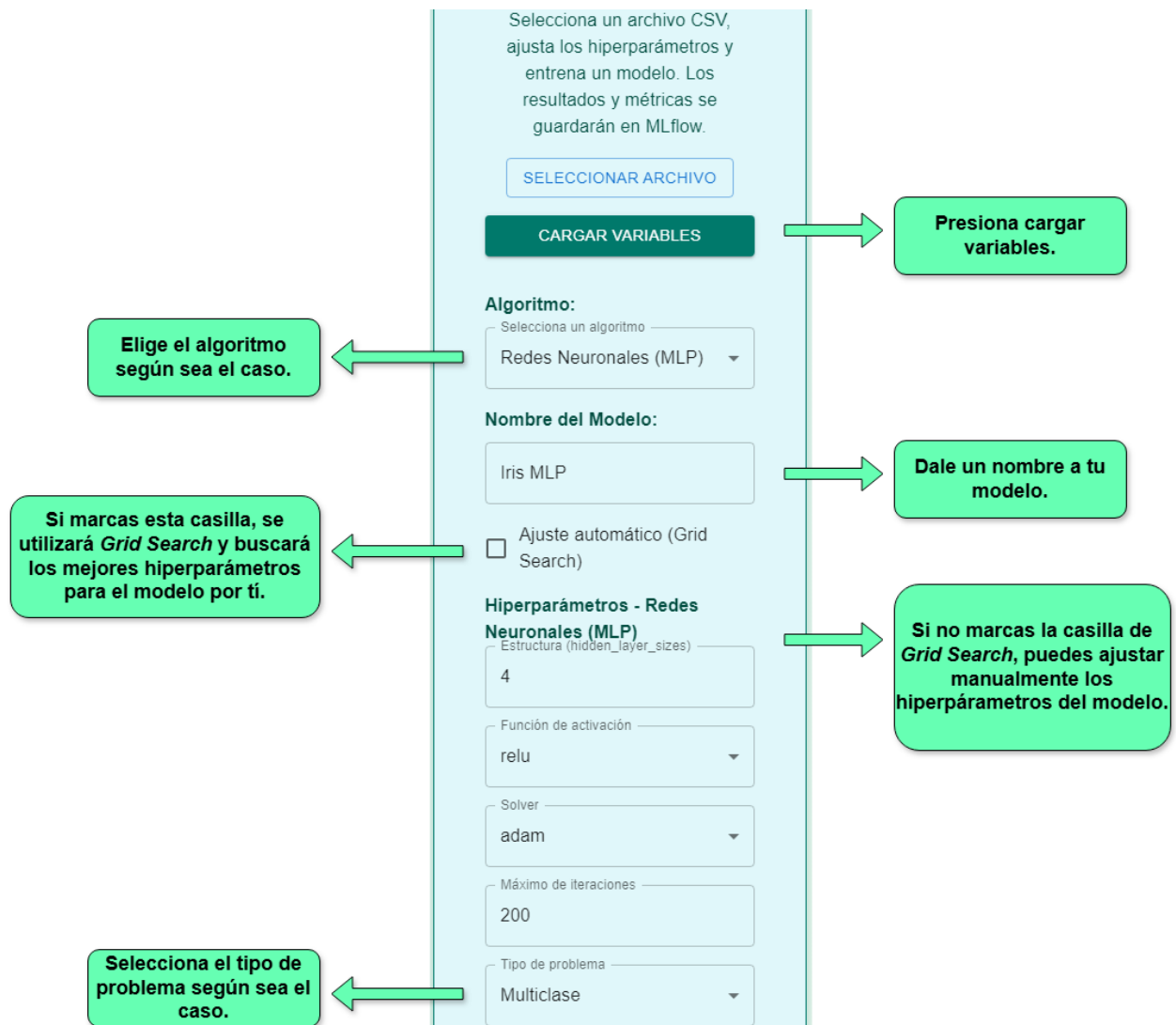
Algoritmo	Tipo de problema	Entrada(Features)	Salida(Targets)
Regresión logística	Clasificación binaria	Numéricas y <code>get_dummies</code> para variables categóricas	1 columna con variables 0/1
Redes neuronales multi perceptron(MLP)	Clasificación binaria	Numéricas y <code>get_dummies</code> para variables categóricas	1 columna con variables 0/1
Redes neuronales multi perceptron(MLP)	Clasificación multiclase	Numéricas y <code>get_dummies</code> para variables categóricas	1 columna con strings o <i>Label Encoder</i>
XGBoost	Clasificación binaria	Numéricas y <code>get_dummies</code> para variables categóricas	1 columna con valores 0/1
XGBoost	Clasificación multiclase	Numéricas y <code>get_dummies</code> para variables categóricas	1 columna con <i>Label Encoder</i>

Una vez realizada la codificación, el dataset codificado se guarda y se versiona con *DVC*, se registran parámetros y métricas en *MLflow* y se actualiza el archivo de configuración del pipeline. El archivo queda guardado bajo el nombre ***processed_train*** en la carpeta ***processed*** del directorio del experimento.

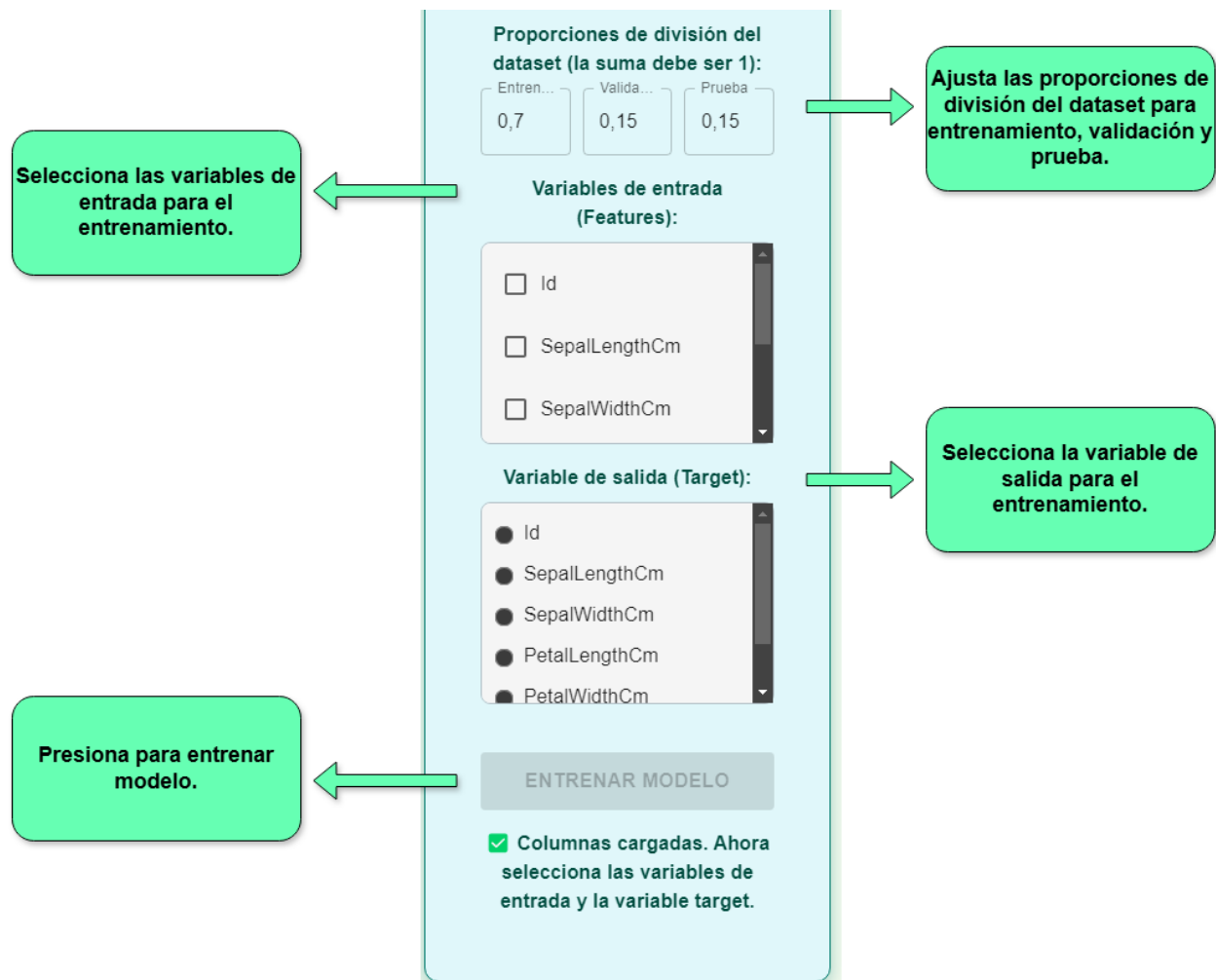
	.gitignore	21-02-2025 22:01	Archivo de origen ...	1 KB
	processed_eda_iris.csv	21-02-2025 20:30	Archivo de valores...	5 KB
	processed_eda_iris.csv.dvc	21-02-2025 20:30	Archivo DVC	1 KB
	processed_eda_iris_cleaning_report.json	21-02-2025 20:30	Archivo de origen ...	1 KB
	processed_train_processed_eda_iris.csv	21-02-2025 22:01	Archivo de valores...	4 KB
	processed_train_processed_eda_iris.csv.dvc	21-02-2025 22:01	Archivo DVC	1 KB

6.4 Entrenamiento y validación

Al presionar **Entrenar Modelo**, el sistema inicia un *run* en MLflow para ejecutar el proceso de entrenamiento y validación. Durante este paso, se guarda el dataset de entrenamiento en un directorio **trained** y se versiona con DVC y Git.



A continuación, se registra el dataset en MLflow y se divide en conjuntos de *entrenamiento*, *validación* y *test*.



Dependiendo del algoritmo seleccionado (regresión logística, MLP o XGBoost), el sistema entrena el modelo usando los hiperparámetros proporcionados o mediante **grid search** y evalúa el desempeño en el conjunto de validación y test, generando métricas (*accuracy*, *f1*, *precisión*, *recall*, *ROC-AUC*, etc.) y gráficos (curvas ROC y matrices de confusión).

Finalmente, el modelo entrenado se registra en *MLflow*, se guarda localmente y se actualiza el archivo ***pipeline_config.json*** con todos los metadatos y rutas de artefactos.

Experiment_2025-02-24_15-36-00_ae014739-5f96-4712-8d73-9f530d65629f >

tasteful-fawn-551

Overview Model metrics System metrics Artifacts

Description 

Entrenamiento mlp

Details

Created at	2025-02-24 17:10:41
Created by	root
Experiment ID	6 
Status	 Finished
Run ID	eef7ae754bc449c19396ce05cd5f4cb3 
Duration	13.6s
Datasets used	 Dataset de Entrenamiento (8dc7d4a3) 
Tags	training_phase: completed 
Source	 __main__.py
Logged models	 sklearn
Registered models	 Iris MLP 

Incluyendo sus parámetros:

Parameters (10)

 Search parameters

Parameter	Value
step	mlp_training
input_features	['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
target_variable	Species
split_ratios	{'train': 0.7, 'val': 0.15, 'test': 0.15}
use_grid_search	True
problem_type	multiclass
best_activation	relu
best_hidden_layer_sizes	(100, 50, 10)
best_max_iter	300
best_solver	sgd

También sus métricas:

Metrics (18)	
Search metrics	
Metric	Value
System metrics (8)	
system/cpu_utilization_percentage	0.2
system/system_memory_usage_meg...	3002.5
system/system_memory_usage_perce...	14.4
system/disk_usage_percentage	4.3
system/disk_usage_megabytes	44285.7
system/disk_available_megabytes	981823.2
system/network_receive_megabytes	0
system/network_transmit_megabytes	0
Model metrics (10)	
val_accuracy	1
val_f1	1
val_precision	1
val_recall	1
test_accuracy	0.9565217391304348
test_f1	0.9602339181286549
test_precision	0.9583333333333334
test_recall	0.9666666666666667
energy_consumed_total_kWh	0.000055443416149595785

Y el modelo registrado:

mlp_model

Iris MLP, v1
Registered on 2025/02/24

Path: file:///app/experimentos/Experiment_2025-02-24_15-36-00_ae014739-5f96-4712-8d73-9f530d65629f/artifacts/ee7ae754bc449c19396ce05cd5f4cb3/artifacts/mlp_model

MLflow Model

The code snippets below demonstrate how to make predictions using the logged model. This model is also registered to the [model registry](#).

Model schema
Input and output schema for your model. [Learn more](#)

Name	Type
Inputs (4)	
Sepal.LengthCm (required)	double
Sepal.WidthCm (required)	double
Petal.LengthCm (required)	double
Outputs (1)	

Make Predictions
Predict on a Spark DataFrame:

```

import mlflow
from pyspark.sql.functions import struct, col
logged_model = 'runs:/ee7ae754bc449c19396ce05cd5f4cb3/mlp_model'

# Load model as a Spark UDF. Override result_type if the model does not return double values.
loaded_model = mlflow.pyfunc.spark_udf(spark, model_uri=logged_model, result_type='double')

# Predict on a Spark DataFrame.
df.withColumn('predictions', loaded_model(struct(*map(col, df.columns))))

```

Predict on a Pandas DataFrame:

```

import mlflow
logged_model = 'runs:/ee7ae754bc449c19396ce05cd5f4cb3/mlp_model'

# Load model as a PyFuncModel.
loaded_model = mlflow.pyfunc.load_model(logged_model)

# Predict on a Pandas DataFrame.
import pandas as pd

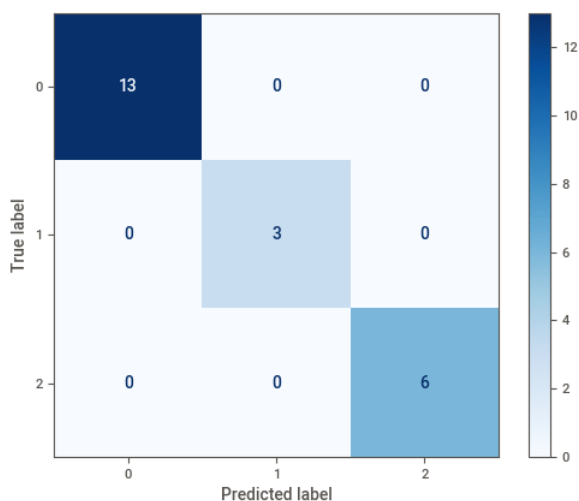
```


Puedes ver en la carpeta **trained** el **dataset** utilizado para el entrenamiento, también el modelo entrenado y las matrices de confusión generadas.

Nombre	Fecha de modificación	Tipo	Tamaño
.dvc	24-02-2025 12:36	Carpeta de archivos	
.dvc_cache	24-02-2025 12:37	Carpeta de archivos	
.git	24-02-2025 17:10	Carpeta de archivos	
artifacts	24-02-2025 17:10	Carpeta de archivos	
eda_reports	24-02-2025 12:38	Carpeta de archivos	
processed	24-02-2025 16:50	Carpeta de archivos	
raw	24-02-2025 12:37	Carpeta de archivos	
trained	24-02-2025 17:10	Carpeta de archivos	
.dvcignore	24-02-2025 12:36	Archivo DVCIGNO...	1 KB
.gitignore	24-02-2025 17:10	Archivo de origen ...	1 KB
confusion_matrix_mlp_test	24-02-2025 17:10	Archivo PNG	13 KB
confusion_matrix_mlp_validation	24-02-2025 17:10	Archivo PNG	13 KB
Iris MLP.pkl	24-02-2025 17:10	Archivo PKL	154 KB
Iris MLP.pkl	24-02-2025 17:10	Archivo DVC	1 KB
pipeline_config	24-02-2025 17:10	Archivo de origen ...	6 KB
pipeline_config.json	24-02-2025 16:50	Archivo DVC	1 KB

En una *matriz de confusión*, cada fila representa la clase real y cada columna la clase predicha. En donde en este ejemplo:

- 13 instancias fueron correctamente clasificadas como 0
- 3 instancias fueron correctamente clasificadas como 1
- 6 instancias fueron correctamente clasificadas como 2



Todo esto permite una trazabilidad completa del proceso de entrenamiento.

6.5 Generar reporte del experimento.

Al presionar **Generar Resumen**, el sistema utiliza el archivo **pipeline_config.json** para crear un *PDF* que resume todo el experimento. Este incluye una tabla con sus parámetros, métricas y rutas de artefactos.



Resumen del Experimento

Experiment ID: ae014739-5f96-4712-8d73-9f530d65629f

Paso 1: data_cleaning

Parámetro	Valor
step	data_cleaning
run_id	dd469ff34d4342d78c8956922789a402
inputs	{}
outputs	<pre>{ "raw_data": { "path": "raw/Iris.csv", "dvc_file": "raw/Iris.csv.dvc" }, "processed_eda": { "path": "processed/processed_eda_Iris.csv", "dvc_file": "processed/processed_eda_Iris.csv.dvc" } }</pre>
parameters	<pre>{ "eliminar_duplicados": false, "filtrar_outliers": false, "relleno_valores_numericos": "dejar", "valor_imputacion": null }</pre>

Este reporte permite obtener una vista consolidada de todos los procesos y resultados obtenidos durante el experimento, facilitando su análisis y documentación.

Guía de uso: Pipeline automatizado

7.1 Ejecución del pipeline

Al presionar Ejecutar Pipeline, el sistema orquesta de forma secuencial todos los pasos definidos en el archivo **pipeline_config.json** usando *DVC* y *MLflow*, reproduciendo todo en un nuevo experimento y se genera un resumen automático que consolida toda la información del flujo.

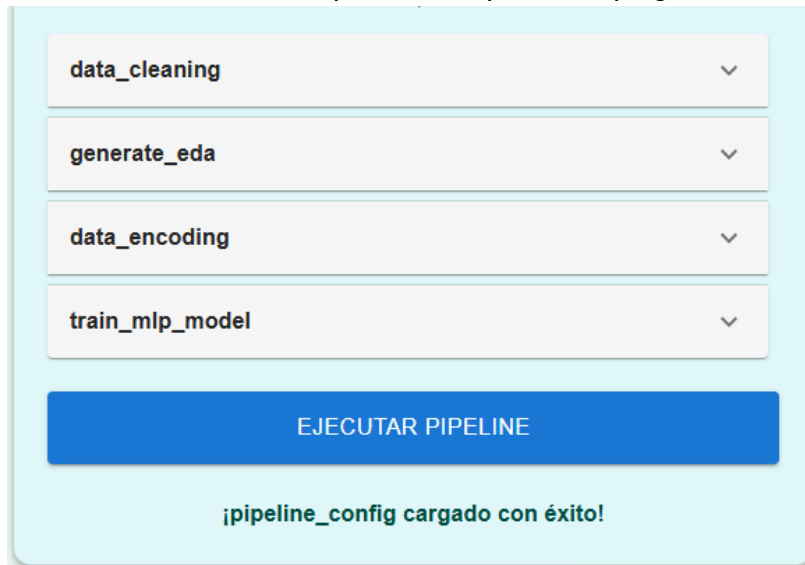


The interface is titled "Ejecución Automática del Pipeline". It contains a text input field for "Ruta base (base_dir):" with the value "/app/experimentos". Below this is a section for "Archivo pipeline_config.json:" with a button "Seleccionar archivo" and the text "Ningún archivo seleccionado". To the right of this is a purple button labeled "CARGAR PIPELINE". At the bottom is a large blue button labeled "EJECUTAR PIPELINE".

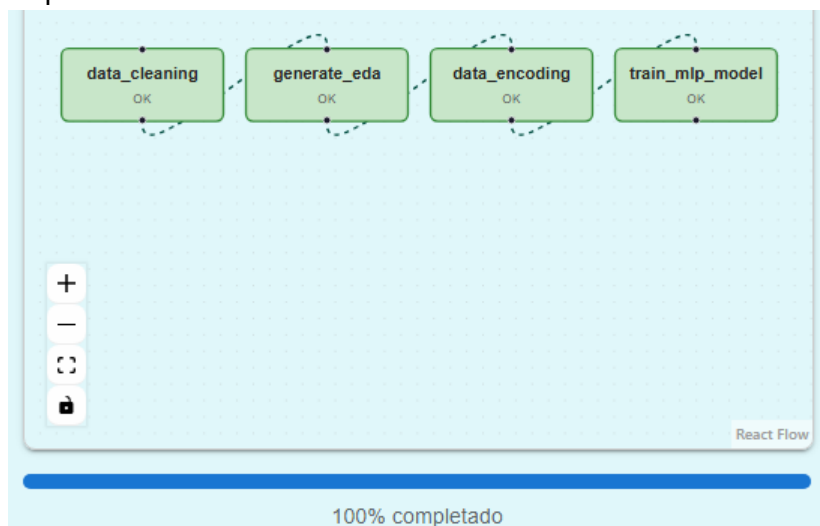
En la interfaz se muestra un diagrama del pipeline cargado con el archivo **pipeline_config.json**:



También puedes ver los detalles de cada paso en un panel desplegable:



Una vez presionas **Ejecutar pipeline**, durante la ejecución, se puede observar en tiempo real el avance de cada paso hasta la finalización de este.



Puedes revisar en tu directorio de experimentos que el experimento fue creado y reproducido exitosamente con cada paso definido en el archivo de configuración y su reporte PDF generado:

	.dvc	24-02-2025 17:53	Carpeta de archivos	
	.dvc_cache	24-02-2025 17:53	Carpeta de archivos	
	.git	24-02-2025 17:54	Carpeta de archivos	
	artifacts	24-02-2025 17:54	Carpeta de archivos	
	eda_reports	24-02-2025 17:54	Carpeta de archivos	
	processed	24-02-2025 17:54	Carpeta de archivos	
	raw	24-02-2025 17:53	Carpeta de archivos	
	trained	24-02-2025 17:54	Carpeta de archivos	
	.dvcignore	24-02-2025 17:53	Archivo DVCIGNO...	1 KB
	.gitignore	24-02-2025 17:54	Archivo de origen ...	1 KB
	confusion_matrix_mlp_test	24-02-2025 17:54	Archivo PNG	13 KB
	confusion_matrix_mlp_validation	24-02-2025 17:54	Archivo PNG	13 KB
	DefaultMLPModel.pkl	24-02-2025 17:54	Archivo PKL	154 KB
	DefaultMLPModel.pkl	24-02-2025 17:54	Archivo DVC	1 KB
	experiment_summary	24-02-2025 17:54	Microsoft Edge P...	8 KB
	pipeline_config	24-02-2025 17:54	Archivo de origen ...	6 KB
	pipeline_config.json	24-02-2025 17:54	Archivo DVC	1 KB

Glosario

- **Experimento:** conjunto de ejecuciones (*runs*) que documentan un flujo de trabajo completo, desde la ingesta y limpieza de datos hasta el entrenamiento y validación del modelo y generación de reportes, permitiendo la trazabilidad de cada paso.
- **MLflow:** plataforma para el registro y seguimiento de experimentos. Permite documentar configuraciones, parámetros, métricas, artefactos y comparar distintos runs mediante su interfaz gráfica.
[Documentación oficial de MLflow](#)
- **DVC:** herramienta que gestiona el versionado de datasets y modelos.
[Documentación oficial de DVC](#)
- **Pipeline:** conjunto de pasos secuenciales que conforman el flujo de trabajo del experimento. El pipeline se configura mediante un archivo *JSON* (*pipeline_config.json*).
- **Run:** ejecución individual registrada en *MLflow*, que documenta métricas, parámetros y resultados de cada paso del pipeline.
- **Reporte del experimento:** documento PDF generado automáticamente que resume todos los pasos, parámetros y resultados del experimento, facilitando su revisión y análisis.

Contacto

Si tienes alguna duda, sugerencia o necesitas asistencia con el sistema, no dudes en comunicarte:

- **Correo electrónico:** leonardo.espinoza.o@usach.cl
- **Teléfono:** +56 9 7371 1546